Western Engineering

Electrical and Computer Engineering

AISE 4030 Reinforcement Learning

Assignment 02: Policy Gradient Methods

Released on: Friday, March 27th, 2026
Due on: Wednesday, April 8th, 2026

Note: Due date includes the 48-hour grace period.

Table of Contents

1. Assignment Overview 1
2. Objectives 1
3. Environment: BipedalWalker 1
   3.1. Installation 1
      3.1.1. Step 1: Create the Conda Environment 1
      3.1.2. Step 2: Install Dependencies 1
      3.1.3. Step 3: Verify the Installation 2
      3.1.4. Step 4: Run Training 2
   3.2. Environment Specifications 2
4. Assignment Tasks 3
   4.1. Shared Components 3
      4.1.1. Gaussian Policy Network (Actor) 3
      4.1.2. Value Network (Critic / Baseline) 3
   4.2. Task 1: REINFORCE with Baseline 3
      4.2.1. Episode Collection 3
      4.2.2. Discounted Returns 3
      4.2.3. Baseline Subtraction 3
      4.2.4. Parameter Updates 4
   4.3. Task 2: Proximal Policy Optimization (PPO) 4
      4.3.1. Trajectory Rollout 4
      4.3.2. Generalized Advantage Estimation (GAE) 4
      4.3.3. Clipped Surrogate Objective 4
      4.3.4. Combined Loss and Update Procedure 5
   4.4. Task 3: Comparative Analysis 5
      4.4.1. Required Plots 5
      4.4.2. Required Discussion 5
5. Code Structure & Naming Conventions 6
   5.1. File Naming and Responsibilities 6
   5.2. Required Directory Structure 7
   5.3. Config-Driven Agent Selection 7
   5.4. Required Components 7
6. Documentation Requirements 8
7. Training Guidelines 8
8. AI Usage Policy 9
9. Submission Details 10

10. Grading Criteria 10
11. Team Collaboration 10

1. Assignment Overview

This assignment introduces you to policy gradient methods, a fundamentally different approach to reinforcement learning from the value-based methods you implemented in Assignment 1. Rather than learning a value function and deriving a policy from it, policy gradient methods directly optimize a parameterized policy by following the gradient of expected return.

You will implement and compare two policy gradient algorithms on the BipedalWalker-v3 environment, a continuous control task where a two-legged robot must learn to walk across randomly generated terrain:

- **REINFORCE with Baseline:** A Monte Carlo policy gradient method that uses a learned value function as a baseline to reduce variance in the gradient estimates.
- **Proximal Policy Optimization (PPO):** A modern actor-critic method that collects fixed-length trajectory rollouts, computes advantages via Generalized Advantage Estimation (GAE), and performs multiple epochs of mini-batch updates using a clipped surrogate objective.

Both algorithms operate in a continuous action space, which is something DQN cannot handle. By holding the environment, network architecture, and core hyperparameters constant, you can isolate and observe how the optimization strategy affects learning stability, sample efficiency, and final performance.

2. Objectives

Upon successful completion of this assignment, you will be able to:

- Understand the policy gradient theorem and how it enables direct optimization of a stochastic policy.
- Implement a Gaussian policy for continuous action spaces, including action sampling and log-probability computation.
- Implement the REINFORCE algorithm with a learned baseline (value function) to reduce variance.
- Understand Monte Carlo return estimation and why high variance is a fundamental challenge in REINFORCE.
- Implement Proximal Policy Optimization (PPO) with the clipped surrogate objective.
- Implement Generalized Advantage Estimation (GAE) and understand how it trades off bias and variance.
- Understand why PPO uses multiple epochs of mini-batch updates on the same rollout data, and how the clipping mechanism prevents destructive policy updates.
- Design controlled experiments and produce comparative analyses supported by specific evidence from training curves.

3. Environment: BipedalWalker

3.1. Installation

Prerequisites: Anaconda or Miniconda must be installed on your machine. If you do not have it, download it from: https://docs.anaconda.com/miniconda/install/

3.1.1. Step 1: Create the Conda Environment

Open a terminal (or Anaconda Prompt on Windows) and run:

```
conda create -n AISE4030_A2 python=3.10 -y
```

Then activate it:

```
conda activate AISE4030_A2
```

You should see (`AISE4030_A2`) at the beginning of your terminal prompt.

3.1.2. Step 2: Install Dependencies

Navigate to your project folder and install all required packages:

```
cd path/to/your-project-folder
pip install -r requirements.txt
```

Your `requirements.txt` must include the following:

```
torch
gymnasium[box2d]
numpy
matplotlib
pyyaml
```

Important: The gymnasium[box2d] extra installs the Box2D physics engine required for BipedalWalker. If you encounter build issues with Box2D on Windows, install SWIG first:

```
conda install swig -y
pip install gymnasium[box2d]
```

3.1.3. Step 3: Verify the Installation

Run the following command to confirm everything is working:

```
python -c "
import gymnasium as gym
env = gym.make('BipedalWalker-v3')
obs, info = env.reset(seed=42)
print('Environment created successfully!')
print('Observation shape:', obs.shape)
print('Action space:', env.action_space)
print('Action range:', env.action_space.low, 'to', env.action_space.high)
env.close()
print('Setup is complete!')
"
```

You should see output similar to:

```
Environment created successfully!
Observation shape: (24,)
Action space: Box(-1.0, 1.0, (4,), float32)
Action range: [-1. -1. -1. -1.] to [1. 1. 1. 1.]
Setup is complete!
```

Note: Include this console output in your report as confirmation that your environment is set up and running.

3.1.4. Step 4: Run Training

Once verified, you can start training with:

```
python training_script.py
```

3.2. Environment Specifications

Property	Value
Environment	BipedalWalker-v3 (via Gymnasium). A 2D bipedal robot on procedurally generated terrain. Each episode generates new terrain, requiring the agent to generalize.
Observation	A 24-dimensional continuous vector containing: hull angle and angular velocity, horizontal and vertical speed, joint angles and angular velocities for each of the 4 joints (hip and knee for each leg), leg ground contact indicators, and 10 lidar rangefinder readings for terrain detection. All values are continuous floats.
Action Space	Box(-1.0, 1.0, (4,), float32). Four continuous values controlling torque applied to the two hip joints and two knee joints. Each action dimension is bounded to [-1, 1].
Reward	Positive reward for moving forward (proportional to horizontal velocity). Small negative cost per step for applying torque (encourages efficiency). A reward of -100 for falling (hull contacts the ground). Total reward for a successful walk to the end of the terrain is approximately +300.
Episode Termination	The episode ends when the robot falls (hull contacts the ground), reaches the far end of the terrain, or 1600 time steps have elapsed.
Solved Threshold	The environment is considered solved at an average reward of +300 over 100 consecutive episodes. For this assignment, achieving a stable average reward above +200 is a strong result.

4. Assignment Tasks

4.1. Shared Components

Both tasks share the same network architecture and action sampling mechanism. Implement these once and reuse them across both agents.

4.1.1. Gaussian Policy Network (Actor)

Your policy_network.py must implement a Gaussian (Normal) policy suitable for continuous action spaces. The network takes a state as input and outputs the parameters of a multivariate Gaussian distribution over actions:

- **Mean head:** A fully connected layer that outputs a mean vector μ(s) with one value per action dimension (4 values for BipedalWalker). Apply tanh activation to bound the mean to [-1, 1], matching the action space bounds.
- **Log standard deviation:** A learnable parameter vector log σ (not state-dependent) with one value per action dimension. This is a common design choice that simplifies training. Initialize log σ to 0.0, corresponding to an initial standard deviation of 1.0.

The network must provide two methods:

- **Forward pass:** Given a state, return the mean and standard deviation of the action distribution.
- **Action sampling:** Sample an action from the Gaussian distribution using the reparameterization trick `(torch.distributions.Normal).Clamp` the sampled action to [-1, 1] before returning. Also return the log-probability of the sampled action.
- **Architecture:** Use two hidden layers of 64 units each with tanh activation for the shared body. The hidden layer sizes and activation function should be configurable via `config.yaml`.

4.1.2. Value Network (Critic / Baseline)

Your `value_network.py` must implement a state-value function V(s) that estimates the expected return from a given state. This network serves as the baseline for REINFORCE and the critic for PPO.

- **Architecture:** Two hidden layers of 64 units each with tanh activation. A single scalar output with no activation. The architecture should mirror the policy network body, but with a separate set of parameters.

4.2. Task 1: REINFORCE with Baseline

Implement the REINFORCE algorithm with a learned baseline in reinforce_agent.py. This is a Monte Carlo policy gradient method: the agent collects complete episodes before performing any parameter updates.

4.2.1. Episode Collection

At each time step within an episode, the agent:

1. Observes the current state s.
2. Samples an action a from the Gaussian policy $\pi_\theta(a|s)$.
3. Records the log-probability log $\pi_\theta(a|s)$, the state s, and the reward r.
4. Repeats until the episode terminates.

No parameter updates occur during the episode. All updates happen after the episode is complete.

4.2.2. Discounted Returns

After the episode ends, compute the discounted return Gt for each time step t:

$$G_t = r_t + \gamma\, r_{t+1} + \gamma^2\, r_{t+2} + \cdots + \gamma^{T-t-1}\, r_{T-1}$$

where T is the episode length, and γ is the discount factor. Normalize the returns across the episode (subtract mean, divide by standard deviation plus a small epsilon) to improve gradient stability.

4.2.3. Baseline Subtraction

For each state st visited during the episode, compute the advantage estimate:

$$A_t = G_t - V(s_t)$$

Where $V(s_t)$ is the output of the value network. Subtracting the baseline does not change the expected gradient direction, but it significantly reduces the variance of the gradient estimates, leading to more stable learning.

4.2.4. Parameter Updates

After computing advantages for all time steps in the episode, perform a single gradient update for both networks:

- **Policy loss:** The negative of the mean of $(\log \pi_\theta(a_t|s_t) \times A_t)$ over all time steps. Gradient ascent on expected return is equivalent to gradient descent on this negated quantity.
- **Value loss:** Mean squared error between the predicted values $V(s_t)$ and the actual returns G_t.

Use separate optimizers for the policy and value networks.

Expected behavior: REINFORCE should learn to walk, but progress will be slow and noisy. Because the gradient is estimated from entire episodes using Monte Carlo returns, the variance is high. Expect large fluctuations in the reward curve even after the agent begins to show competence. This motivates the more sample-efficient approach in Task 2.

4.3. Task 2: Proximal Policy Optimization (PPO)

Implement the PPO algorithm with the clipped surrogate objective in ppo_agent.py. PPO is an actor-critic method that collects fixed-length trajectory rollouts, computes advantages using Generalized Advantage Estimation, and performs multiple mini-batch gradient updates per rollout using a clipped objective to prevent destructive policy changes.

4.3.1. Trajectory Rollout

Instead of waiting for a complete episode, PPO collects a fixed number of transitions (the rollout length) by interacting with the environment. For each time step in the rollout, record:

- The state s_t.
- The action a_t is sampled from the current policy.
- The log-probability $\log \pi_{\theta_{old}}(a_t|s_t)$ under the current policy.
- The reward r_t.
- The value estimate $V(s_t)$ from the critic.
- The done flag indicates if the episode terminated.

If an episode terminates mid-rollout, reset the environment and continue collecting transitions until the rollout buffer is full.

4.3.2. Generalized Advantage Estimation (GAE)

After collecting a rollout, compute advantages using GAE. For each time step t, the GAE advantage is defined recursively:

$$\delta_t = r_t + \gamma\, V(s_{t+1}) - V(s_t)$$
$$A_t^{\text{GAE}} = \delta_t + (\gamma\lambda)\delta_{t+1} + (\gamma\lambda)^2\delta_{t+2} + \cdots$$

where δ_t is the one-step TD error, γ is the discount factor, and λ is the GAE parameter controlling the bias-variance tradeoff. When $\lambda = 1$, GAE reduces to Monte Carlo advantage estimation (high variance, low bias). When $\lambda = 0$, it reduces to one-step TD advantage (low variance, high bias). Use $\lambda = 0.95$ as the default.

Compute the GAE advantages in reverse order (from the last time step to the first) for efficiency. Normalize the advantages across the rollout batch (subtract mean, divide by standard deviation) before using them in the loss computation.

The return targets for the value function are computed as:

$$R_t = A_t^{\text{GAE}} + V(s_t)$$

4.3.3. Clipped Surrogate Objective

PPO reuses the same rollout data for multiple gradient updates by optimizing a clipped surrogate objective. For each transition in a mini-batch, compute the probability ratio:

$$r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$$

In practice, compute this ratio in log-space for numerical stability:

$$r_t(\theta) = exp\big(\log \pi_\theta(a_t|s_t) - \log \pi_{\theta_{old}}(a_t|s_t)\big)$$

The clipped surrogate loss is:

$$L^{\text{CLIP}} = -E[min(r_t(\theta)A_t,\ \text{clip}(r_t(\theta), 1-\epsilon\,, 1+\epsilon)A_t)]$$

where ϵ is the clipping parameter (default 0.2). This objective prevents the policy from changing too much in a single update: if the ratio moves outside the interval $[1-\epsilon, 1+\epsilon]$, the gradient is clipped. This is the core mechanism that makes PPO stable.

4.3.4. Combined Loss and Update Procedure

The total PPO loss combines three terms:

- **Clipped surrogate loss (actor):** As defined above.
- **Value function loss (critic):** Mean squared error between $V(s_t)$ and the return targets R_t, scaled by a coefficient c_1 (default 0.5).
- **Entropy bonus:** The mean entropy of the policy distribution, scaled by a coefficient c_2 (default 0.01). This encourages exploration by penalizing policies that become too deterministic too quickly. The entropy of a Gaussian distribution is $0.5 \times \ \ln(2\pi e\sigma^2)$, summed across action dimensions.

$$\text{Total Loss} = L^{\text{CLIP}} + c_1 \times L^{\text{VF}} - c_2 \times H(\pi)$$

Update procedure: For each collected rollout, perform K epochs (default 10) of mini-batch updates. In each epoch, shuffle the rollout data and split it into mini-batches. For each mini-batch, recompute log-probabilities under the current policy, compute the ratio and clipped loss, and update both actor and critic using a single shared optimizer.

Expected behavior: PPO should learn faster and more stably than REINFORCE. The GAE advantage estimates have lower variance than Monte Carlo returns, and the clipped objective allows multiple gradient steps per rollout, making PPO significantly more sample-efficient. Expect a smoother, steadily climbing reward curve compared to the noisy REINFORCE baseline.

4.4. Task 3: Comparative Analysis

After training both agents, produce a written analysis comparing their performance. This section is graded on the quality of your reasoning about your own experimental results. Generic explanations copied from textbooks or AI tools will not receive full marks; you must reference specific behaviors observed in your training curves.

4.4.1. Required Plots

Generate the following plots and include them in your report:

1. **Reward Curve (both agents overlaid):** Episode reward (moving average) over training episodes for both algorithms on the same plot.
2. **Loss Curves (both agents overlaid):** Policy loss and value loss over training for both algorithms.
3. **Per-agent detailed plots:** Individual reward and loss plots for each agent showing raw episode rewards and the moving average.
4. **Entropy plot (PPO only):** Mean policy entropy over training, showing how exploration evolves.

4.4.2. Required Discussion

Your written analysis must address the following questions. For each, cite specific evidence from your plots:

1. **Convergence Comparison:** Which agent converged faster? Which achieved a higher final reward? At approximately what episode did each agent begin to show consistent improvement?
2. **Stability Analysis:** Compare the variance in each agent's reward curve. Which showed more stable learning? Did either agent exhibit reward regression after reaching a high point?
3. **Sample Efficiency:** REINFORCE discards data after a single update; PPO reuses each rollout across multiple epochs. How does this difference manifest in the learning curves? How many total environment steps did each agent require to reach a comparable reward level?
4. **Variance Reduction:** Both methods use a value function baseline. However, REINFORCE uses full Monte Carlo returns while PPO uses GAE. How does the choice of an advantage estimator affect the noise in the gradient and the smoothness of learning?

5. **Practical Reflection:** Based on your results, in what scenario would you choose REINFORCE over PPO, if any? What tradeoffs does PPO introduce (e.g., additional hyperparameters, implementation complexity)?

5. Code Structure & Naming Conventions

Your project must be organized into modular, single-responsibility files. The table below describes the required file structure, the naming convention for each file, and its responsibility within the project.

5.1. File Naming and Responsibilities

Why this naming matters: Algorithm-specific naming makes it immediately clear which file belongs to which implementation. The two agents share the same `policy_network.py`, `value_network.py`, `environment.py`, `config.yaml`, `training_script.py`, and `utils.py`. The only files that differ between agents are the agent class itself (`reinforce_agent.py` vs. `ppo_agent.py`) and the rollout buffer (used by PPO only). This mirrors the structure from Assignment 1, where all three D3QN variants shared the same network and environment code.

File	Naming Rule	Responsibility
`config.yaml`	Fixed name	All hyperparameters, environment settings, agent type selection, file paths, and execution modes. Avoids hard-coding values in Python files.
`environment.py`	Fixed name	Environment creation and any wrappers or preprocessing. Returns the environment instance, observation shape, and action space dimensions.
`policy_network.py`	Named after network type	Gaussian policy network (actor). Outputs mean and standard deviation for the action distribution. Provides action sampling and log-probability computation. Shared by both agents.
`value_network.py`	Named after network type	State-value function network (critic / baseline). Outputs a single scalar V(s). Used as the baseline in REINFORCE and as the critic in PPO. Shared by both agents.
`reinforce_agent.py`	Named after algorithm	REINFORCE with Baseline agent: episode collection, Monte Carlo return computation, baseline subtraction, policy gradient update, and value function update.
`ppo_agent.py`	Named after algorithm	PPO agent: rollout collection, GAE computation, clipped surrogate objective, entropy bonus, multi-epoch mini-batch updates, and value function update.
`rollout_buffer.py`	Named after buffer type	On-policy trajectory storage for PPO. Stores states, actions, log-probabilities, rewards, values, and done flags for a fixed-length rollout. Provides mini-batch sampling for multi-epoch updates.
`training_script.py`	Fixed name	Main entry point: reads `config.yaml`, instantiates the appropriate agent, runs the training loop, logs metrics, saves results, and supports evaluation/deployment mode.
`utils.py`	Fixed name	Shared utilities: config loading, plotting, logging, seed management, and helper functions.
`README.md`	Fixed name	Project overview, installation and setup instructions, how to run each agent, and configuration reference.
`requirements.txt`	Fixed name	All Python dependencies with pinned versions where necessary.
`reinforce_results/` `ppo_results/`	Named after algorithm	Output directories for saved model weights, training history (CSV or JSON), and generated plots. One directory per agent.

5.2. Required Directory Structure

```
BipedalWalker_PolicyGradient/
├── config.yaml              # All hyperparameters and settings
├── environment.py            # BipedalWalker environment setup
├── policy_network.py         # Gaussian policy network (actor, shared)
├── value_network.py          # Value function network (critic, shared)
├── reinforce_agent.py        # REINFORCE with Baseline agent (Task 1)
├── ppo_agent.py              # PPO agent with clipped objective (Task 2)
├── rollout_buffer.py         # On-policy trajectory storage (PPO only)
├── training_script.py        # Main entry point: training and evaluation
├── utils.py                  # Config loading, plotting, logging, helpers
├── README.md                 # Project overview, setup, how to run
├── requirements.txt          # Python dependencies
├── reinforce_results/        # Task 1 outputs (model, history, plots)
└── ppo_results/              # Task 2 outputs (model, history, plots)
```

5.3. Config-Driven Agent Selection

The `config.yaml` must control which agent variant is trained. Your `training_script.py` should read an `agent_type` parameter from the config and instantiate the appropriate agent and buffer. A single training script should be able to run both experiments by changing only the config file. Example:

```yaml
# config.yaml
agent_type: "ppo"    # Options: "reinforce", "ppo"

environment:
  name: "BipedalWalker-v3"
  seed: 42

shared:
  gamma: 0.99
  hidden_sizes: [64, 64]
  activation: "tanh"
  max_grad_norm: 0.5

reinforce:
  policy_lr: 0.0003
  value_lr: 0.001
  max_episodes: 3000

ppo:
  lr: 0.0003
  rollout_length: 2048
  mini_batch_size: 64
  epochs: 10
  clip_epsilon: 0.2
  gae_lambda: 0.95
  value_loss_coef: 0.5
  entropy_coef: 0.01
  max_episodes: 2000
```

5.4. Required Components

Regardless of the agent, your implementation must include the following core components. The exact method names may vary, but these responsibilities must be clearly present and documented in your code.

Component	Description
Initialization	Constructor that accepts hyperparameters from config.yaml, creates the policy and value networks, sets up optimizers, and initializes any storage structures.

Action Selection	Method to sample an action from the Gaussian policy given a state. Must return the action and its log-probability. Should support a deterministic mode for evaluation (using the mean action without sampling).
Learning / Update	Method to update the policy and value network parameters. For REINFORCE: compute discounted returns, compute advantages, perform a single policy gradient step. For PPO: compute GAE advantages, run K epochs of mini-batch updates with clipped objective.
Trajectory Storage	For REINFORCE: internal lists to store episode transitions (states, actions, log-probs, rewards), cleared after each update. For PPO: a rollout buffer class that stores fixed-length trajectories and provides shuffled mini-batch iteration.
Return / Advantage Computation	For REINFORCE: method to compute discounted Monte Carlo returns and subtract the baseline. For PPO: method to compute GAE advantages and return targets using TD errors.
Save / Load Model	Methods to persist and restore both policy and value network weights for checkpointing and deployment.
Training Loop	A main training function (in training_script.py) that runs episodes, calls the agent, logs metrics (reward, loss, entropy), saves checkpoints, and writes results to the appropriate results directory.
Evaluation / Deployment	A function that loads a trained model and runs it in deterministic mode (mean action, no sampling) for evaluation. Should support rendering the environment for visual inspection.

6. Documentation Requirements

Each class and method in your code must include a complete Python docstring that describes its purpose, parameters (with types), and return values (with types). This applies to all files, not just agent classes.
Example of required documentation style:

```
def compute_gae(rewards, values, dones, next_value, gamma, lam):
    """
    Computes Generalized Advantage Estimation (GAE) for a rollout.

    Args:
        rewards (list[float]): Rewards collected during the rollout.
        values (list[float]): Value estimates V(s) for each state.
        dones (list[bool]): Terminal flags for each step.
        next_value (float): Bootstrap value V(s') for the last state.
        gamma (float): Discount factor.
        lam (float): GAE lambda parameter.

    Returns:
        tuple: (advantages, returns)
            - advantages (np.ndarray): GAE advantage estimates.
            - returns (np.ndarray): Discounted return targets for
              value function training.
    """
```

7. Training Guidelines

The following tables provide recommended hyperparameters for each agent. You may adjust these based on your experimentation, but you must document any changes and justify them in your report.

Shared Hyperparameters

Hyperparameter	Value	Notes
Total training episodes	2,000 - 5,000	Adjust based on available compute
Discount factor (γ)	0.99	Standard for continuous control tasks
Hidden layer sizes	[64, 64]	Two hidden layers, both actor and critic
Activation function	tanh	Standard for policy gradient methods
Gradient clipping	0.5	Max gradient norm

REINFORCE with Baseline

Hyperparameter	Value	Notes
Policy learning rate	3e-4	Adam optimizer for the policy network
Value learning rate	1e-3	Adam optimizer for the value network
Update frequency	Every episode	Monte Carlo: update after each complete episode

PPO

Hyperparameter	Value	Notes
Learning rate	3e-4	Single Adam optimizer for both actor and critic
Rollout length	2048	Transitions collected per rollout
Mini-batch size	64	Rollout data split into mini-batches
Number of epochs (K)	10	Gradient updates per rollout
Clip parameter (ϵ)	0.2	Surrogate objective clipping range
GAE λ	0.95	Bias-variance tradeoff in advantage estimation
Value loss coefficient (c_1)	0.5	Scales critic loss relative to actor loss
Entropy coefficient (c_2)	0.01	Encourages exploration; decay to 0 is optional

Important: Both agents must be trained with the same shared hyperparameters (discount factor, network sizes, activation, gradient clipping) to ensure a fair comparison. The only variables between experiments should be the policy optimization algorithm and its specific hyperparameters.

8. AI Usage Policy

You are permitted and encouraged to use AI tools (e.g., ChatGPT, Claude, GitHub Copilot, Gemini, etc.) to assist you throughout this assignment. However, transparency and accountability are required.
You must include an AI Usage Log as an appendix to your submission. This log must contain:

- **Tool Identification:** The name and version of each AI tool used.
- **Purpose:** A brief description of how the tool was used.
- **Full Prompt Log:** A complete list of all prompts you submitted, in order. You may paraphrase very long conversations, but the substance of every prompt must be captured.
- **What You Changed:** Whether you used the AI output as-is or modified it, and what changes you made.

Academic Integrity:
Using AI without documenting it in your AI Usage Log will be treated as a violation of academic integrity. You are fully responsible for understanding and being able to explain every line of code you submit. You may be asked to explain any part of your implementation during grading.

9. Submission Details

You must submit two (2) items to the assignment portal:

1. PDF Report (3-5 pages): A report containing:
   - Confirmation that your environment is set up and running (include console output showing observation space, action space, and device).
   - Training results: all required plots from Section 4.4.1.
   - Comparative analysis addressing all questions from Section 4.4.2.
   - A brief description of any hyperparameter changes you made and why.
   - AI Usage Log (as described in Section 8), included as an appendix.
   - Team Contribution Efforts: Each member must outline their specific contributions in short bullet points.
2. Code Archive (`.zip` file): A single `.zip` file containing:
   - All code files following the structure in Section 5.
   - `config.yaml` with all hyperparameters used.
   - README.md with setup instructions and how to run each agent.
   - `requirements.txt` listing all Python dependencies.
   - Results directories with saved models, training histories, and plots.

10. Grading Criteria

Criteria	Description	Points
Task 1: REINFORCE with Baseline	Correct Gaussian policy implementation, Monte Carlo return computation, baseline subtraction, and policy gradient update. Code is well-documented and runs without errors.	**25**
Task 2: PPO	Correct rollout collection, GAE computation, clipped surrogate objective, entropy bonus, and multi-epoch mini-batch updates. This is the most technically demanding task.	**30**
Comparative Analysis	All required plots are included. Discussion is specific to the team's own results, references concrete evidence from training curves, and demonstrates understanding of why each algorithm behaves as observed.	**25**
Code Organization	File structure follows Section 5 conventions. Includes `config.yaml`, `README.md`, and `requirements.txt`. All classes and methods have complete docstrings.	**10**
AI Usage Log	Complete and honest documentation of AI tool usage, including all prompts and modifications.	**5**
Professionalism	Report is clear, well-written, and properly formatted. Team contributions are included.	**5**
Total		**100**

11. Team Collaboration

This is a team-based assignment using the same teams as your course project. All team members are expected to contribute. Your report must include a Team Contribution Efforts section in which each member outlines their specific contributions in short bullet points. Failure to include this section will result in a deduction under the Professionalism criterion.